CHULETA FINAL DE JAVASCRIPT

2o DAW - Repaso completo + examen 2_2 | Metodos, sintaxis, patrones y errores tipicos

Basada en los materiales de clase aportados y en el ejercicio 2_2 (tienda.js).

OBJETIVO: Reconocer rapidamente que metodo, estructura o patron necesita cada enunciado, y escribir una solucion segura sin modificar datos cuando el ejercicio lo prohiba.

Como usar esta chuleta

- Primero mira el verbo del enunciado: transformar, filtrar, buscar, comprobar, calcular, ordenar, anadir o eliminar.
- Despues decide si necesitas un valor, un elemento, una posicion, un array nuevo o modificar el array original.
- Por ultimo comprueba mutabilidad, tipos y el valor de retorno del metodo.
- En el ejercicio 2_2, los patrones de Partes 4-6 son prioritarios.

INDICE RAPIDO POR TIPO

Tipo	Metodo	Muta?
Anadir al final	push(...)	Modifica
Quitar del final	pop()	Modifica
Anadir al inicio	unshift(...)	Modifica
Quitar del inicio	shift()	Modifica
Borrar/reemplazar	splice(i,n,...)	Modifica
Copiar parte	slice(i,f)	No
Unir arrays	concat(...)	No
Recorrer sin crear array	forEach(fn)	No
Transformar	map(fn)	No
Filtrar	filter(fn)	No
Calcular/acumular	reduce(fn, inicial)	No
Reducir derecha->izquierda	reduceRight(fn,inicial)	No
Buscar posicion	indexOf(x)	No
Buscar ultima posicion	lastIndexOf(x)	No
Buscar existencia	includes(x)	No
Buscar elemento	find(fn)	No
Buscar posicion por criterio	findIndex(fn)	No
Buscar ultima posicion por criterio	findLastIndex(fn)	No
Comprobar alguno	some(fn)	No
Comprobar todos	every(fn)	No
Ordenar numeros	sort((a,b)=>a-b)	Modifica
Ordenar texto	sort((a,b)=>a.localeCompare(b))	Modifica
Invertir	reverse()	Modifica

Tipo	Metodo	Muta?
Rellenar	fill(v)	Modifica
Aplanar	flat(n)	No
Aplanar+transformar	flatMap(fn)	No
String -> array	split(sep)	No
Array -> string	join(sep)	No
Comprobar array	Array.isArray(v)	No

MAPA DE DECISION: QUE METODO ELEGIR

Enunciado habitual	Eleccion	Esqueleto
"Transforma cada elemento"	map	array.map(x => ...)
"Quedate con los que cumplen"	filter	array.filter(x => condicion)
"Haz algo con cada uno"	forEach	array.forEach(x => ...)
"Encuentra el primero"	find	array.find(x => condicion)
"Dime su indice"	findIndex	array.findIndex(x => condicion)
"Existe este valor"	includes	array.includes(valor)
"Alguno cumple"	some	array.some(x => condicion)
"Todos cumplen"	every	array.every(x => condicion)
"Calcula un total"	reduce	array.reduce((acc,x)=>...,0)
"Ordena"	sort	[...array].sort(comparador) si no se puede mutar
"Primero de la cola"	shift	cola.shift()
"Urgente al principio"	unshift	cola.unshift(x)
"Ultimo de la pila"	pop	pila.pop()
"Anade al carrito"	push	carrito.push(x)
"Elimina por posicion"	indexOf + splice	const i=array.indexOf(x); array.splice(i,1)
"Copia un tramo"	slice	array.slice(inicio,fin)
"Texto separado por..."	split	texto.split(separador)
"Une elementos con..."	join	array.join(separador)
"Arrays anidados"	flat / flatMap	array.flat() / array.flatMap(fn)

1. FUNDAMENTOS QUE TIENES QUE TENER AUTOMATICOS

Variables, tipos y conversiones

Tema	Sintaxis / ejemplo	Recuerda
let	let edad = 18; edad = 19;	Puede reasignarse.
const	const nombre = "Miku";	No puede reasignarse; arrays/objetos pueden mutar.
typeof	typeof 42 // "number"	typeof null es "object" por una particularidad historica.
Array.isArray	Array.isArray(valor)	Es la comprobacion correcta para arrays.

Tema	Sintaxis / ejemplo	Recuerda
undefined	let x; // undefined	Ausencia de valor no inicializado.
null	let x = null;	Ausencia intencionada.
Number	Number("12") // 12	Conversion a numero.
parseInt	parseInt("12") // 12	Entero.
parseFloat	parseFloat("3.14") // 3.14	Decimal.
toString	(12).toString() // "12"	Numero -> string.

Strings y template literals

```
const nombre = "Miku"
nombre.length
const mensaje = `Producto: ${nombre}`
```

REGLA: Para construir mensajes del ejercicio usa preferentemente `...${...}...`; evita concatenaciones largas con +.

Objetos, destructuring y JSON

Concepto	Sintaxis	Uso
Objeto	const p = { nombre: "Zelda", precio: 60 };	Agrupar datos relacionados.
Propiedad	p.nombre / p[clave]	Acceder a datos.
Anadir	p.stock = 3	Agregar propiedad.
Borrar	delete p.stock	Eliminar propiedad.
Copia con cambio	{ ...p, stock: 2 }	Crear objeto nuevo y sobreescribir una propiedad.
Destructuring objeto	const { nombre, precio } = p	Extraer propiedades.
Destructuring array	const [a,b] = valores	Extraer por posicion.
Rest	const [primero, ...resto] = valores	Recoger el resto en un array.
JSON -> JS	JSON.parse(texto)	Convertir texto JSON a dato JS.
JS -> JSON	JSON.stringify(objeto)	Convertir dato JS a texto JSON.

2. ESTRUCTURAS DE CONTROL

Comparaciones y logica

Sintaxis	Significado	Ejemplo
===	Igualdad estricta	"42" === 42 // false
!==	Desigualdad estricta	stock !== 0
< > <= >=	Comparaciones	precio >= 50
&&	Y	stock > 0 && precio > 0
||	O	vip || estudiante
!	Negacion	!activo
??	Valor por defecto si null/undefined	nombre ?? "Sin nombre"
?.	Acceso seguro	usuario?.direccion?.calle

EXAMEN: Usa === y !==. Evita == y !=. Para NaN, usa Number.isNaN(x).

if / else / ternario / switch

```
if (stock > 0) {
  estado = "Disponible"
} else {
  estado = "Agotado"
}

const estado2 = stock > 0 ? "Disponible" : "Agotado"

switch (opcion) {
  case 1: ...; break
  case 2: ...; break
  default: ...
}
```

Bucles

Estructura	Cuando	Esqueleto
while	Repetir mientras se cumpla una condicion	while (condicion) { ... }
do...while	Necesitas al menos una vuelta	do { ... } while (condicion)
for	Controlar un indice/contador	for (let i=0; i<arr.length; i++) { ... }
for...of	Recorrer valores	for (const x of array) { ... }
for...in	Recorrer claves	for (const clave in objeto) { ... }
break	Salir del bucle	if (cond) break
continue	Saltar a siguiente vuelta	if (cond) continue

REGLA: for...of -> valores. for...in -> claves. En Part 6.1, piensa while + shift para procesar una cola.

3. FUNCIONES Y PROGRAMACION FUNCIONAL

Funciones

```
function sumar(a, b) {
  return a + b
}

const sumar2 = (a, b) => a + b
const doble = x => x * 2
const saludo = () => "Hola"
```

Concepto	Que significa	Ejemplo
Parametro	Nombre que recibe la funcion	function f(x) {} -> x
Argumento	Valor que pasas al llamar	f(10) -> 10
return	Devuelve y termina la funcion	return resultado
Flecha sin llaves	Return implicito	x => x * 2

Concepto	Que significa	Ejemplo
Flecha con llaves	Necesita return	x => { return x * 2 }
Valor por defecto	Se usa si llega undefined	(x = 0) => x + 1
Rest	Recoge argumentos	(primero, ...resto) => ...
Spread	Expande un array	Math.max(...numeros)

IDEA CLAVE: map, filter, find, some, every y reduce reciben funciones. Las funciones son valores de primera clase.

4. METODOS DE ARRAYS - FICHAS DE EXAMEN

push(...) - AGREGAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.push(...items)	Anadir al final.	Nueva longitud	SI

```
carrito.push("Zelda")
```

USO CONJUNTO: push -> cola; push + shift = FIFO.

ERROR TIPICO: Confundir push con unshift.

pop() - QUITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.pop()	Quitar el ultimo.	Elemento eliminado / undefined	SI

```
const ultimo = pila.pop()
```

USO CONJUNTO: push + pop = pila LIFO.

ERROR TIPICO: No usar para quitar el primero.

unshift(...) - AGREGAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.unshift(...items)	Anadir al principio.	Nueva longitud	SI

```
cola.unshift(pedidoUrgente)
```

USO CONJUNTO: unshift + pop tambien puede implementar una cola.

ERROR TIPICO: Usarlo cuando el enunciado diga urgente/primero.

shift() - QUITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.shift()	Quitar el primero.	Elemento eliminado / undefined	SI

```
const siguiente = cola.shift()
```

USO CONJUNTO: En 5.1 y 6.1: cola FIFO.

ERROR TIPICO: No confundir con pop().

splice(i,n,...) - EDITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.splice(indice, cuantos, ...nuevos)	Borrar, insertar o reemplazar en el medio.	Array con lo eliminado	SI

```
carrito.splice(posicion, 1)
arr.splice(2, 0, "nuevo")
```

USO CONJUNTO: indexOf + splice = eliminar la primera aparicion.

ERROR TIPICO: splice modifica el original; slice no.

slice(i,f) - COPIAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.slice(inicio, fin)	Copiar un tramo o hacer una copia completa.	Array nuevo	NO

```
const copia = array.slice()
const trozo = array.slice(1, 3)
```

USO CONJUNTO: slice + sort permite ordenar una copia.

ERROR TIPICO: El fin NO se incluye.

concat(...) - UNIR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.concat(...items)	Unir arrays sin mutar.	Array nuevo	NO

```
const todos = a.concat(b)
```

USO CONJUNTO: Alternativa: [...a, ...b].

ERROR TIPICO: concat no modifica los originales.

5. METODOS DE BUSQUEDA

indexOf(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.indexOf(valor)	Saber la primera posicion de un valor concreto.	Indice o -1	NO

```
const i = carrito.indexOf("Zelda")
```

USO CONJUNTO: indexOf + splice para borrar una aparicion.

ERROR TIPICO: Busca por valor concreto; usa find para objetos/criterios.

lastIndexOf(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.lastIndexOf(valor)	Saber la ultima posicion de un valor concreto.	Indice o -1	NO

```
historial.lastIndexOf("añadir")
```

USO CONJUNTO: Muy util en deshacer cuando hay repetidos.

ERROR TIPICO: No devuelve el elemento.

includes(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.includes(valor)	Saber solo si un valor existe.	true / false	NO

```
["Ana","Luis"].includes("Luis")
```

USO CONJUNTO: Tambien puede comprobar existencia tras map.

ERROR TIPICO: No sirve para buscar con una condicion.

find(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.find(x => condicion)	Encontrar el primer elemento que cumple.	Elemento o undefined	NO

```
catalogo.find(p => p.nombre === "Zelda")
```

USO CONJUNTO: find + propiedad; find para luego leer/modificar un objeto encontrado.

ERROR TIPICO: Puede devolver undefined: no accedas a propiedades sin comprobar.

findIndex(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.findIndex(x => condicion)	Encontrar la posicion del primer elemento que cumple.	Indice o -1	NO

```
const i = catalogo.findIndex(p => p.nombre === "Zelda")
```

USO CONJUNTO: findIndex + splice para eliminar por criterio.

ERROR TIPICO: Si no existe, devuelve -1.

findLastIndex(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.findLastIndex(x => condicion)	Buscar la ultima posicion que cumple un criterio.	Indice o -1	NO

```
historial.findLastIndex(x => x.tipo === "quitar")
```

USO CONJUNTO: Alternativa especializada cuando se quiere la ultima coincidencia.

ERROR TIPICO: Es una extension de 02.2, no el patron central de tienda.js.

6. TRANSFORMAR, FILTRAR Y CALCULAR

map(fn) - TRANSFORMAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.map(x => resultado)	Crear un array nuevo transformando cada elemento.	Array nuevo del mismo tamaño	NO

```
catalogo.map(p => p.nombre)
```

USO CONJUNTO: filter(...).map(...), map(...).join("\n"), map(...).flat()

ERROR TIPICO: No es para filtrar: siempre genera una salida por elemento.

filter(fn) - FILTRAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.filter(x => condicion)	Quedarte con todos los elementos que cumplen.	Array nuevo	NO

```
catalogo.filter(p => p.stock > 0)
```

USO CONJUNTO: filter + map es el patron mas tipico.

ERROR TIPICO: No devuelve un unico objeto.

forEach(fn) - RECORRER

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.forEach((x,i,arr) => ...)	Ejecutar una accion por elemento sin crear resultados.	undefined	NO*

```
productos.forEach(p => console.log(p.nombre))
```

USO CONJUNTO: Usalo cuando el resultado es un efecto (console.log, push, etc.).

ERROR TIPICO: Si necesitas un array, usa map; forEach devuelve undefined.

reduce(fn,inicial) - CALCULO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reduce((acc, x) => ..., inicial)	Acumular hasta obtener un unico valor.	Un valor	NO

```
const total = productos.reduce((s,p) => s + p.precio, 0)
```

USO CONJUNTO: reduce + find; reduce para totales; reduce con objeto acumulador.

ERROR TIPICO: Especifica valor inicial; sin el, [] produce error.

reduceRight(fn,inicial) - CALCULO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reduceRight((acc,x) => ..., inicial)	Reducir empezando por la derecha.	Un valor	NO

```
["a","b","c"].reduceRight((t,x) => t+x, "")
```

USO CONJUNTO: 02.2 lo presenta como variante de reduce.

ERROR TIPICO: No confundir con reverse + reduce.

some(fn) - COMPROBACION

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.some(x => condicion)	Comprobar si al menos uno cumple.	true / false	NO

```
catalogo.some(p => p.stock === 0)
```

USO CONJUNTO: some = "hay alguno"; ideal para hayAgotados.

ERROR TIPICO: No significa todos.

every(fn) - COMPROBACION

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.every(x => condicion)	Comprobar si todos cumplen.	true / false	NO

```
pedido.lineas.every(linea => linea.cantidad > 0)
```

USO CONJUNTO: every = "todos"; clave en puedeServirse y preciosValidos.

ERROR TIPICO: Asegurate de recorrer el array correcto.

7. ORDENAR Y METODOS EXTRA

sort(fn) - ORDENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.sort((a,b) => ...)	Ordenar elementos.	Array ordenado	SI

```
[...productos].sort((a,b) => a.precio - b.precio)
```

USO CONJUNTO: Si el enunciado no permite mutar: [...array].sort(...).

ERROR TIPICO: Sin comparador, los numeros se comparan como texto.

localeCompare - ORDENAR TEXTO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
a.localeCompare(b)	Ordenar cadenas respetando el idioma/acento.	Numero <0, 0, >0	NO

```
productos.sort((a,b) => a.nombre.localeCompare(b.nombre))
```

USO CONJUNTO: sort + localeCompare para nombres.

ERROR TIPICO: No uses a > b si el ejercicio habla de orden alfabetico con acentos.

reverse() - ORDENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reverse()	Invertir el orden.	Array invertido	SI

```
[...array].reverse()
```

USO CONJUNTO: sort ascendente + reverse para descendente, si el ejercicio lo plantea.

ERROR TIPICO: Tambien muta: copia primero si hace falta.

fill(v,i,f) - RELLENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.fill(valor, inicio, fin)	Rellenar posiciones.	Array	SI

```
new Array(5).fill("■")
```

USO CONJUNTO: En graficoStock: new Array(n).fill("■").join("").

ERROR TIPICO: Modifica el array.

flat(n) - APLANAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.flat(profundidad)	Aplanar arrays anidados.	Array nuevo	NO

```
[["A","B"],["C"]].flat()
```

USO CONJUNTO: map + flat aparece en cadenas de procesamiento.

ERROR TIPICO: flat() por defecto aplana un nivel.

flatMap(fn) - APLANAR + TRANSFORMAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.flatMap(x => resultadoArray)	Transformar y aplanar un nivel en una sola operacion.	Array nuevo	NO

```
["hola mundo"].flatMap(x => x.split(" "))
```

USO CONJUNTO: Alternativa compacta a map(...).flat().

ERROR TIPICO: Aplana un solo nivel.

split(sep) - STRING -> ARRAY

Sintaxis	Cuando usarlo	Que devuelve	Muta?
texto.split(separador)	Separar una cadena por un delimitador.	Array	NO

```
"Ana|Pan:2;Leche:1".split("|")
```

USO CONJUNTO: 4.1: split("|") -> split(";") -> split(":").

ERROR TIPICO: split siempre trabaja sobre strings.

join(sep) - ARRAY -> STRING

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.join(separador)	Unir elementos en un texto.	String	NO

```
["Pan","Leche"].join("\n")
```

USO CONJUNTO: 4.5: map + join("\n") para generar ticket.

ERROR TIPICO: No confundir con split.

Array.isArray(v) - TIPO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
Array.isArray(valor)	Comprobar si un valor es un array.	true / false	NO

```
Array.isArray(matriz)
```

USO CONJUNTO: 1.1 y validaciones.

ERROR TIPICO: typeof [] devuelve "object".

toSorted(fn) - ORDENAR SIN MUTAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.toSorted(comparador)	Ordenar sin tocar el original (ES2023).	Array nuevo	NO

```
const ordenados = numeros.toSorted((a,b) => a-b)
```

USO CONJUNTO: Alternativa moderna a [...array].sort(...).

ERROR TIPICO: Tu ejercicio 2_2 trabaja con ES6 y puede exigir spread/slice.

8. TABLA VITAL: MUTA O NO MUTA

Categoria	Metodos / patron
SI mutan	push, pop, shift, unshift, splice, sort, reverse, fill, copyWithin
NO mutan	slice, concat, map, filter, reduce, reduceRight, find, findIndex, findLastIndex, indexOf, lastIndexOf, includes, some, every, join, flat, flatMap
Copia tipica	[...array] o array.slice()
Objeto copia	{ ...objeto, propiedad: nuevoValor }

EXAMEN 2_2: Excepto Parte 5, las funciones no deben modificar los arrays de entrada. Para ordenar, copia antes con [...array] o slice().

9. CUANDO USAR METODOS JUNTOS: RECETAS

Combinacion	Idea	Patron	Uso en examen
filter + map	Seleccionar y despues transformar	productos.filter(p => p.stock > 0).map(p => p.nombre)	Nombres de productos con stock.
map + join	Transformar y construir texto	lineas.map(l => `${l.nombre}: ${l.cantidad}`).join("\n")	Ticket / listado.
split + map	Parsear una cadena a datos	texto.split(";").map(x => x.split(":"))	Pedidos.
find + propiedad	Buscar un objeto y usar un dato	const p = catalogo.find(x => x.nombre === nombre); p.precio	Buscar precio/stock.
findIndex + splice	Eliminar el primer elemento que coincide	const i = a.findIndex(x => x.id === id); if (i !== -1) a.splice(i,1)	Borrar por criterio.
indexOf + splice	Eliminar un valor concreto	const i = a.indexOf(valor); if (i !== -1) a.splice(i,1)	Carrito Parte 5.
reduce + find	Acumular usando datos de objetos	lineas.reduce((t,l) => t + buscarProducto(catalogo,l.nombre).precio*l.cantidad, 0)	Total de pedido.
map + flat + filter + indexOf + sort	Transformar, aplanar, quitar repetidos y ordenar	pedidos.map(p => p.lineas).flat().map(l => l.nombre).filter((n,i,a) => a.indexOf(n) === i).sort((a,b)=>a.localeCompare(b))	productosVendidos 6.2.
[...] + sort	Ordenar sin mutar	[...catalogo].sort((a,b) => a.precio-b.precio)	ordenarPorPrecio / 3 mas baratos.
while + shift	Procesar una cola hasta vaciarla	while (cola.length > 0) { const p = cola.shift(); ... }	procesarCola 6.1.
new Array + fill + join	Crear una barra visual	new Array(Math.floor(stock/2)).fill("■").join("")	graficoStock 6.3.

10. EJERCICIO 2_2 - GUIA DE PATRONES POR PARTES

PRIORIDAD MAXIMA: Partes 4, 5 y 6. Aprende el patron, no memorices la solucion completa.

Parte 4 - Pedidos

Funcion	Que pide	Patron mental	Metodos clave
4.1 parsearPedido	Texto -> objeto pedido	capas de split + map + Number	split, map, Number
4.2 puedeServirse	Todos existen y tienen stock suficiente	pedido.lineas.every -> buscarProducto -> stock >= cantidad	every, find
4.3 totalPedido	Total euros	reduce -> buscarProducto -> precio*cantidad	reduce, find
4.4 servirPedido	Nuevo catalogo con stock descontado	catalogo.map -> find linea -> { ...producto, stock: ... }	map, find, spread
4.5 generarTicket	Texto final con saltos de linea	map -> template literal -> join("\n")	map, join, template literal

```
// 4.1 esquema
const partes = texto.split("|")
const cliente = partes[0]
const lineas = partes[1].split(";").map(linea => {
  const p = linea.split(":")
  return { nombre: p[0], cantidad: Number(p[1]) }
})
```

```
// 4.2 esquema
return pedido.lineas.every(linea => {
  const producto = buscarProducto(catalogo, linea.nombre)
  return producto !== undefined && producto.stock >= linea.cantidad
})

// 4.3 esquema
return pedido.lineas.reduce((total, linea) => {
  const producto = buscarProducto(catalogo, linea.nombre)
  return total + producto.precio * linea.cantidad
}, 0)
```

Parte 5 - Arrays mutables

Funcion	Idea	Metodo	Patron
5.1 atenderSiguiente	Sale el primero	shift	const pedido = cola.shift()
5.2 agregarUrgente	Entra por delante	unshift	cola.unshift(pedido)
5.3 agregarAlCarrito	Entra al final + historial	push	carrito.push(producto)
5.4 quitarDelCarrito	Buscar primera posicion y borrar	indexOf + splice	const i = carrito.indexOf(x); if (i !== -1) carrito.splice(i,1)
5.5 deshacer	Deshacer ultima accion	pop + reverse action + lastIndexOf/splice	const accion = historial.pop(); ...

PISTA: Parte 5 trata al historial como una pila LIFO: la ultima accion registrada es la primera que se deshace.

Parte 6 - Integracion

Funcion	Secuencia	Clave
6.1 procesarCola	while -> shift -> puedeServirse -> servirPedido -> reasignar catalogo	Cola + condicion + inmutabilidad
6.2 productosVendidos	map -> flat -> map -> filter + indexOf -> sort + localeCompare	Pipelines encadenados
6.3 graficoStock	calcular n -> new Array(n).fill("■").join("")	String visual desde un array

```
// 6.1 patron
while (cola.length > 0) {
  const pedido = atenderSiguiente(cola)
  if (puedeServirse(catalogo, pedido)) {
    catalogo = servirPedido(catalogo, pedido)
  }
}
```

11. CHECKLIST DE EXAMEN: ANTES DE ENTREGAR

- [ ] He leido exactamente que pide el enunciado y que array debo recorrer.
- [ ] He elegido map/filter/find/findIndex/some/every/reduce segun el verbo.
- [ ] He usado === y !==, no ==/!=.
- [ ] He comprobado undefined cuando find puede no encontrar nada.
- [ ] He comprobado -1 cuando uso indexOf/findIndex antes de splice.
- [ ] He usado Number() cuando un numero viene de split().
- [ ] He usado localeCompare para texto cuando el orden alfabetico importa.
- [ ] He puesto comparador numerico en sort((a,b)=>a-b).
- [ ] He copiado antes de sort/reverse si no puedo mutar la entrada.

- [ ] He recordado que const no congela arrays/objetos.
- [ ] He usado shift para sacar el primero y push/unshift segun donde entra.
- [ ] He distinguido slice (copia) de splice (modifica).
- [ ] He distinguido split (string -> array) de join (array -> string).
- [ ] En flechas con llaves he puesto return.
- [ ] En reduce he puesto un valor inicial, normalmente 0 para sumas.
- [ ] En 4.2 recorro pedido.lineas, no catalogo.
- [ ] En 4.4 creo objetos nuevos con {...producto, stock: ...}.
- [ ] En 6.1 uso while(cola.length > 0).
- [ ] En 6.2 reconozco la cadena map/flat/map/filter/indexOf/sort.
- [ ] No he cambiado nombres ni parametros que el profesor haya fijado.

12. LA SUPER-CHULETA DE 30 SEGUNDOS

Pregunta del enunciado	Respuesta
Transformar cada elemento	map
Seleccionar varios	filter
Hacer algo por cada uno	forEach
Buscar un elemento	find
Buscar una posicion	findIndex
Buscar si existe	includes
Existe alguno	some
Cumplen todos	every
Calcular un total	reduce
Ordenar numeros	sort((a,b)=>a-b)
Ordenar textos	sort((a,b)=>a.localeCompare(b))
Anadir final	push
Quitar final	pop
Anadir inicio	unshift
Quitar inicio	shift
Eliminar por posicion	splice
Copiar	slice / [...array]
Texto -> array	split
Array -> texto	join
Array -> array plano	flat
Copia objeto + cambio	{ ...objeto, prop: valor }
Todos los pedidos hasta vaciar cola	while + shift
Ultima accion primero	pop (LIFO)
Primero que llega primero sale	shift (FIFO)

MIKU: No intentes recordar 50 soluciones distintas. Reconoce 15-20 patrones y combina los metodos que ya conoces.

Documento de estudio elaborado a partir de los materiales aportados para el repaso del examen.